06
PHASE

Data Engineering Fundamentals

Build Reliable Pipelines.
Move Data. Power Insights.

Build Strong Pipelines. Enable Better Decisions.

Learnlytics
.handbook
LEARN SMART. ANALYZE BETTER. GROW FASTER.

Unit 1: Data Integration (ELT & ETL)

1.1 ETL vs. ELT: Why Modern Cloud Stacks Prefer Extract-Load-Transform

Definition

ETL stands for Extract, Transform, Load. It is the traditional way of moving data: you pull data from a source, clean/reshape it, and then load it into a destination database.

ELT stands for Extract, Load, Transform. In this newer approach, you first load raw data into the destination (usually a cloud data warehouse like Snowflake or BigQuery), and then transform it there using SQL.

Sub-Topics

Step	ETL	ELT
Extract	Pull data from source	Pull data from source
Transform	Transform BEFORE loading (outside DB)	Transform AFTER loading (inside DB using SQL)
Load	Load clean data into destination	Load raw data into destination first
Engine	Separate ETL server (e.g. Talend)	Cloud warehouse itself (BigQuery, Snowflake)
Speed	Slower for large data	Faster — leverages cloud compute
Cost	Lower storage cost	Storage is cheap; compute scales well

Why Important

- As a data analyst, you work with data warehouses every day. Understanding ELT helps you know why your raw data lands as-is in staging tables.
- ELT means you can write your own SQL transformations using tools like dbt — giving analysts more power without needing a separate engineering team.
- Helps you debug data pipelines — if data is wrong, you know whether the issue is in extraction or transformation.

- ETL is still relevant for legacy systems, regulated industries, or when you need to mask data before it enters the warehouse.

Real-World Analogy

Think of ETL like a restaurant kitchen that prepares (cooks & plates) food in a central factory before shipping it to the restaurant. ELT is like getting raw ingredients delivered directly to the restaurant's kitchen and cooking there — using the restaurant's powerful in-house equipment. Modern cloud kitchens (warehouses) are so powerful that cooking on-site (ELT) is faster and more flexible.

Syntax (ELT with SQL in Warehouse)

```
-- Step 1: Raw data is already loaded into raw_orders table
-- Step 2: Transform using SQL (ELT)
CREATE OR REPLACE TABLE analytics.clean_orders AS
SELECT
 order_id,
 customer_id,
 UPPER(TRIM(customer_name)) AS customer_name,
 CAST(order_date AS DATE) AS order_date,
 order_amount * 83.0 AS order_amount_inr
FROM raw.raw_orders
WHERE order_amount > 0;
```

Example

Scenario: An e-commerce company uses Fivetran to extract orders from their MySQL database and load them directly into BigQuery (ELT). A data analyst then writes a dbt model to clean the data: removing nulls, standardising date formats, and calculating revenue in local currency. The raw table is preserved for debugging, and the transformed table is used for dashboards.

Industry Use Case

Retail (Amazon, Flipkart): ELT pipelines load millions of order records daily into Snowflake. Analysts then transform this raw data into sales summaries, inventory forecasts, and customer segments — all using SQL inside the warehouse, with no separate transformation server.

Banking (HDFC, ICICI): ETL is preferred here because sensitive customer data must be masked and validated before it enters the data warehouse — ensuring compliance with RBI regulations.

1.2 Batch vs. Real-Time Processing: When to Use Which

Definition

Batch Processing: Data is collected over a period of time (e.g., hourly, daily) and processed all at once. Like running payroll at end of month.

Real-Time (Stream) Processing: Data is processed immediately as it arrives — event by event — with no waiting. Like a live cricket scoreboard updating every ball.

Sub-Topics

Factor	Batch	Real-Time / Streaming
Latency	Minutes to hours	Milliseconds to seconds
Complexity	Simple to build	More complex to build
Cost	Lower cost	Higher cost
Tools	Spark, SQL, Airflow, dbt	Kafka, Flink, Spark Streaming, Kinesis
Use When	Historical reports, overnight jobs	Fraud detection, live dashboards
Data Volume	Large historical batches	Continuous small events

Why Important

- Choosing the wrong processing mode leads to either stale data (batch when real-time needed) or unnecessary cost (real-time when batch is enough).
- Data analysts must understand this to set correct expectations on dashboard refresh rates.
- Micro-batch (e.g., every 5 minutes using Spark Structured Streaming) is a middle ground used frequently.

Real-World Analogy

Batch = A newspaper printed once in the morning — complete, but not instant. Real-Time = A news app that sends a notification the moment a story breaks. The choice depends on whether you can wait for news or need it immediately.

Syntax

Batch (Airflow DAG trigger):

```
schedule_interval='@daily' # Run once a day
schedule_interval='0 6 * * *' # Run at 6 AM every day (cron)
```

Real-Time (Kafka consumer concept):

```
consumer.subscribe(['orders-topic'])
for message in consumer:
 process(message.value) # Process each event as it arrives
```

Example

Batch: An HR system processes monthly payroll by aggregating all attendance records, calculating salaries, and generating payslips — all run at month-end as a single batch job.

Real-Time: A food delivery app (like Swiggy) tracks the rider's GPS location and updates it on the customer's screen every 3 seconds — this requires streaming, not batch.

Industry Use Case

E-Commerce: Sales reports, inventory snapshots → Batch (daily). Fraud alerts on payment transactions → Real-Time streaming with Apache Kafka.

Healthcare: Patient records sync → Batch (nightly). ICU vitals monitoring → Real-Time streaming to alert doctors instantly.

1.3 Change Data Capture (CDC): How Databases Sync Only New/Updated Rows

Definition

Change Data Capture (CDC) is a technique that tracks and captures only the rows that have been inserted, updated, or deleted in a source database — instead of copying the entire table every time. It makes data pipelines more efficient by transferring only what changed.

Sub-Topics

Three main CDC methods:

- Timestamp-Based CDC: Queries rows where updated_at > last_run_time. Simple but can miss deletes.
- Log-Based CDC: Reads the database transaction log (binlog in MySQL, WAL in PostgreSQL) to capture all changes in real time. Most reliable method.
- Trigger-Based CDC: Database triggers write changes to a separate audit table. Adds overhead to the source DB.

Popular CDC Tools:

- Debezium — open-source, connects to MySQL/Postgres/MongoDB
- AWS DMS — managed CDC for AWS environments
- Fivetran / Airbyte — handle CDC automatically for supported connectors

Why Important

- Without CDC, pipelines must reload entire tables every run — extremely slow for large tables with millions of rows.
- CDC enables near-real-time data sync without stressing the source system.
- As an analyst, CDC explains why your warehouse data has 'operation' columns like INSERT, UPDATE, DELETE that track record history.

Real-World Analogy

Imagine a library catalogue. Instead of printing a completely new catalogue every week, the librarian only handbook down which books were added, removed, or moved. CDC does the same — it only ships the 'diff' (difference) rather than the full dataset.

Syntax

Timestamp-Based CDC Query:

```
-- Pull only rows updated since the last pipeline run
SELECT *
FROM source_db.orders
WHERE updated_at > '2024-06-01 00:00:00'
 AND updated_at <= '2024-06-02 00:00:00';
```

Merge (Upsert) into Warehouse (Snowflake):

```
MERGE INTO analytics.orders AS target
USING staging.orders_cdc AS source
ON target.order_id = source.order_id
WHEN MATCHED THEN UPDATE SET target.status = source.status
WHEN NOT MATCHED THEN INSERT (order_id, status) VALUES (source.order_id,
source.status);
```


Example

A bank's core system has 50 million customer accounts. Running a full table copy nightly takes 4 hours. After enabling CDC with Debezium reading the PostgreSQL WAL logs, only ~10,000 rows that changed today are synced — taking just 2 minutes. The data warehouse always has a fresh, accurate view.

Industry Use Case

Retail: Track inventory changes in real time — when stock drops below threshold, a CDC event triggers an automatic reorder workflow.

Telecom: Customer plan updates (upgrades, downgrades, churns) captured via CDC feed a real-time churn prediction model.

1.4 Managed Connectors: Using Fivetran, Airbyte, or Stitch to Automate Ingestion

Definition

Managed connectors are pre-built, ready-to-use data pipeline tools that automatically extract data from a wide range of sources (databases, SaaS apps, APIs) and load it into your data warehouse — without writing custom code. You simply configure source + destination, and they handle scheduling, error handling, and schema changes automatically.

Sub-Topics

Feature	Fivetran	Airbyte	Stitch
Type	Commercial / SaaS	Open-source + Cloud	Commercial / SaaS
Best For	Enterprise, reliability	Flexibility, custom connectors	Small teams, simplicity
Connectors	300+ pre-built	350+ + custom	130+
Pricing	Per MAR (rows)	Open-source free	Per row
CDC Support	Yes (log-based)	Yes	Limited

Why Important

- Saves weeks of engineering time — no need to write, maintain, or debug custom API extractors.
- Automatically handles schema changes (e.g., when Salesforce adds a new field, Fivetran adds the column in your warehouse automatically).
- Analysts get faster access to data from all business tools — CRM, helpdesk, marketing, payments — all in one warehouse.
- Provides data lineage and sync logs — useful for debugging data freshness issues.

Real-World Analogy

Think of managed connectors as professional movers (packers & movers service). Instead of renting a truck, packing everything yourself, and driving — you hire experts who handle the entire process. They know exactly how to pack fragile items (handle schema changes), work on a schedule (automated sync), and notify you if something breaks.

Syntax (Airbyte API — start a sync job)

Fivetran / Airbyte are GUI-configured, but also expose APIs:

```
# Trigger a manual sync via Airbyte REST API
POST /v1/connections/{connectionId}/sync

# Response
{ "job": { "id": "7832", "status": "running" } }
```

Example

A startup uses Fivetran to sync data from Stripe (payments), HubSpot (CRM), and PostgreSQL (app database) into BigQuery every hour. The data team connects all three in a single afternoon using Fivetran's UI — without writing a single line of ingestion code. They then build a unified revenue dashboard in Looker.

Industry Use Case

SaaS Companies: Use Fivetran to centralise data from 10+ tools (Zendesk, Salesforce, Mixpanel, Stripe) into a single Snowflake warehouse — enabling cross-platform analytics like customer health score.

D2C Brands: Stitch syncs Shopify orders, Facebook Ads spend, and Google Analytics data into Redshift — giving analysts a unified view of marketing ROI vs actual sales.

1.5 API Ingestion: Understanding How Data is Pulled from SaaS Tools (Salesforce, Zendesk)

Definition

API Ingestion is the process of extracting data from external applications by calling their Application Programming Interface (API) — a structured way for software to communicate. SaaS tools like Salesforce, Zendesk, HubSpot, and Google Analytics expose REST APIs that your pipeline can query to retrieve data on demand.

Sub-Topics

Key Concepts:

- REST API: The most common type. Uses HTTP methods (GET, POST) and returns data as JSON.
- Authentication: APIs require credentials — usually API Keys, OAuth 2.0 tokens, or username/password (Basic Auth).
- Pagination: Large datasets are split into pages. You must loop through all pages to get complete data (e.g., page=1, page=2...).
- Rate Limiting: APIs restrict how many requests you can make per minute/hour. Exceeding this returns a 429 error.
- Endpoints: Specific URLs that return specific data (e.g., /api/v2/tickets for Zendesk tickets).
- Webhooks: A 'push' alternative — the SaaS app sends data to your URL whenever an event happens (no polling needed).

Why Important

- Most modern business data lives in SaaS tools — CRM data in Salesforce, support tickets in Zendesk, emails in HubSpot. API ingestion is how you get that data into your warehouse.
- Understanding API structure helps analysts troubleshoot when managed connectors fail — you can manually query the API to verify the data.
- Knowing pagination and rate limits explains why some connectors are slow or why data is sometimes missing.

Real-World Analogy

An API is like a restaurant's waiter. You (the pipeline) place an order (API request) from the menu (available endpoints). The waiter (API) goes to the kitchen (database), brings back your dish (JSON data), and if the kitchen is busy (rate limit), makes you wait. Pagination is like ordering a large meal in multiple courses.

Syntax (Python — Zendesk API call)

```
import requests

# Authentication
url = "https://yourcompany.zendesk.com/api/v2/tickets.json"
headers = {"Authorization": "Bearer YOUR_API_TOKEN"}

# Fetch first page
response = requests.get(url, headers=headers)
data = response.json()
tickets = data['tickets']

# Handle pagination
while data.get('next_page'):
 response = requests.get(data['next_page'], headers=headers)
 data = response.json()
 tickets.extend(data['tickets'])
```

Example

A customer success team wants to analyse Zendesk ticket trends. The analyst writes a Python script that calls the Zendesk REST API every night, fetches all new tickets (with pagination), and loads them into BigQuery. They then build a Looker dashboard showing ticket volume by product, priority, and resolution time.

Industry Use Case

CRM Analytics: Salesforce API is called nightly to pull opportunity data (deal stage, owner, amount, close date) into the warehouse — enabling sales performance dashboards without exporting Excel files.

Marketing Agencies: Google Ads and Facebook Ads APIs are called daily to pull spend, impressions, and click data — merged with sales data to calculate true ROAS (Return on Ad Spend).

1.6 File-Based Ingestion: Handling CSV, Parquet, and Avro Files

Definition

File-based ingestion is the process of reading structured or semi-structured data stored in files — such as CSV, Parquet, or Avro — and loading them into a data warehouse or data lake. Files are often deposited in cloud storage (Amazon S3, GCS, Azure Blob) by external partners, legacy systems, or batch export processes.

Sub-Topics — File Format Comparison

Property	CSV	Parquet	Avro
Format Type	Row-based text	Column-based binary	Row-based binary
Human Readable	Yes (open in Excel)	No	No
Compression	None (or gzip)	Excellent (Snappy, GZIP)	Good

Read Speed	Slow for analytics	Very fast for analytics	Fast for full rows
Schema	No built-in schema	Schema embedded	Schema in header (JSON)
Best For	Simple data sharing	Analytics queries	Streaming, Kafka
Storage Size	Large	Small (3-10x smaller)	Medium
Tools Support	Universal	Spark, BigQuery, Athena	Kafka, Spark, Hadoop

Why Important

- CSV is the most common format for data received from vendors, government portals, and legacy systems. Analysts handle CSV files daily.
- Parquet is the standard for modern data lakes — knowing why it's faster helps analysts choose the right format for large-scale analysis.
- Understanding file formats helps in troubleshooting ingestion errors (wrong delimiter, encoding issues, schema mismatches).
- Avro is commonly used in event streaming (Kafka) — relevant when working with real-time data ingestion.

Real-World Analogy

CSV is like a hand-written note — anyone can read it, but searching through 10,000 handbook for a specific word is slow. Parquet is like a well-indexed library book — harder for a human to read directly, but you can find the exact chapter in seconds. Avro is like a sealed envelope with a label — efficient to carry (stream), and the label tells you exactly what is inside.

Syntax

Read CSV in Python (pandas):

```
import pandas as pd
df = pd.read_csv('sales_data.csv', encoding='utf-8', sep=',')
```

Read Parquet in Python (pandas):

```
df = pd.read_parquet('sales_data.parquet')
```

Load CSV into BigQuery (SQL):

```
LOAD DATA INTO myproject.raw.sales
FROM FILES (
 format='CSV',
 uris=['gs://my-bucket/sales_data.csv'],
 skip_leading_rows=1
);
```

Load Parquet into Snowflake (SQL):

```
COPY INTO raw.sales
FROM @my_s3_stage/sales_data.parquet
FILE_FORMAT = (TYPE = PARQUET);
```

Example

A logistics company receives daily delivery reports from 50 city warehouses as CSV files — uploaded to an S3 bucket by 8 AM. An Airflow pipeline picks up all files, converts them to Parquet for efficiency, and loads them into Redshift. Analysts can then run fast SQL queries on 6 months of delivery data thanks to Parquet's columnar storage.

Industry Use Case

Government / Public Sector: Open data portals (data.gov.in) publish CSV files. Analysts download, clean, and load them into their warehouse for policy analysis or visualisation.

Manufacturing: IoT sensors export sensor readings as Avro files to Kafka topics every second. A Flink streaming job reads Avro events and loads aggregated metrics into a time-series database for real-time monitoring.

Insurance: Claims data is shared by partner hospitals as Parquet files on Azure Data Lake. Actuaries use Spark SQL to query 5 years of claims history to calculate risk models.

Quick Reference Summary

Topic	Core Concept	Key Tools	Analyst Takeaway

1.1 ETL vs ELT	ELT preferred in cloud — transform inside warehouse using SQL	dbt, Snowflake, BigQuery	Write SQL transformations; raw data is always preserved
1.2 Batch vs Real-Time	Batch for history; streaming for instant insights	Airflow, Kafka, Flink	Set correct dashboard refresh expectations
1.3 CDC	Sync only changed rows — faster, less load on source	Debezium, Fivetran, DMS	Explains why tables have INSERT/UPDATE/DELETE columns
1.4 Managed Connectors	Pre-built pipelines — no custom code needed	Fivetran, Airbyte, Stitch	Fastest way to get SaaS data into your warehouse
1.5 API Ingestion	Pull data from SaaS apps via HTTP requests	Python requests, Postman	Handle pagination, auth, and rate limits
1.6 File Ingestion	CSV=human-readable; Parquet=analytics-fast; Avro=streaming	pandas, Spark, COPY INTO	Always prefer Parquet for large analytical workloads

Unit 1 — Data Integration (ELT & ETL)

Interview-Base Unit Test— Warmup & Interviewer Levels

Level 1: Warmup Test

5 Multiple Choice Questions | Topic: Data Integration (ELT & ETL)

Q1. What does ELT stand for, and what is its key difference from ETL?

A) Extract, Load, Transform — transformation happens before loading
B) Extract, Load, Transform — transformation happens after loading inside the warehouse
C) Extract, Link, Transform — data is linked between sources first
D) Extract, Layer, Transfer — data is layered in the pipeline before transfer

✓ Answer:

Q2. Which of the following is the BEST reason to use batch processing over real-time streaming?

A) You need to detect credit card fraud the moment a transaction occurs
B) A GPS tracking app needs to update a rider's live location every 3 seconds
C) A payroll system processes employee salary calculations at the end of each month
D) A live sports scoreboard updates after every ball bowled

✓ Answer:

Q3. Which CDC method reads the database transaction log to capture all changes in real time?

A) Timestamp-Based CDC — queries rows where updated_at > last_run_time
B) Trigger-Based CDC — uses database triggers to write changes to an audit table
C) Log-Based CDC — reads the binlog (MySQL) or WAL (PostgreSQL) for all changes
D) Snapshot-Based CDC — takes a full table snapshot and compares differences

✓ Answer:

Q4. Which managed connector is open-source and best known for its flexibility with custom connectors?

A) Fivetran — commercial SaaS focused on enterprise reliability
B) Stitch — designed for simplicity and small teams
C) Airbyte — open-source with 350+ connectors and custom connector support
D) Debezium — a CDC tool that reads database transaction logs

✓ Answer:

Q5. Which file format is BEST suited for large-scale analytics queries due to its columnar storage and excellent compression?

A) CSV — human-readable, universal support
B) Avro — row-based binary format ideal for streaming with Kafka
C) JSON — flexible schema, widely used in APIs
D) Parquet — columnar binary format with excellent compression (3–10x smaller)

✓ Answer:

Level 2: Interviewer Test

15 Intermediate–Advanced Questions | Descriptive · Scenario · Coding · Database

Section A — Descriptive Questions

Descriptive — Q1

Explain the difference between ETL and ELT. Why do modern cloud-based data stacks prefer ELT over traditional ETL?

Answer:

Descriptive — Q2

Compare Batch Processing and Real-Time (Streaming) Processing. What factors should a data team consider when choosing between them?

Answer:

Descriptive — Q3

What is Change Data Capture (CDC)? Describe the three main CDC methods and their trade-offs.

Answer:

Descriptive — Q4

Explain what API Ingestion is. What are the key concepts an analyst must understand when pulling data from a SaaS tool's REST API?

Answer:

Section B — Scenario-Based Descriptive Questions

Scenario — Q1

Scenario: You are a data analyst at an e-commerce startup. Your company uses Shopify for orders, HubSpot for CRM, and a MySQL production database. Your manager asks you to build a single analytics view of customer orders. You have no backend engineering support. What approach would you recommend, and why?

Answer:

Scenario — Q2

Scenario: A bank's nightly ETL job takes 4 hours to copy a 50-million-row customer accounts table from the core banking system into the data warehouse. The team is told to reduce this to under 10 minutes. What solution would you propose, and what are the risks?

Answer:

Scenario — Q3

Scenario: A food delivery company processes 1 million GPS location updates per minute from delivery riders. The analytics team wants to build a live rider tracking dashboard. A junior analyst suggests using a daily batch job. Do you agree? What would you recommend?

Answer:

Scenario — Q4

Scenario: Your company receives daily CSV sales files from 30 regional partners, uploaded to an S3 bucket by 8 AM. The pipeline loads them into Redshift for reporting. A partner complains their data isn't showing up in dashboards. Walk through how you would debug this end-to-end.

Answer:

Section C — Coding Questions

Coding — Q1

Write a Python script to call the Zendesk REST API, retrieve all open tickets with pagination, and store them in a list. Handle authentication with a Bearer token.

Answer:

Coding — Q2

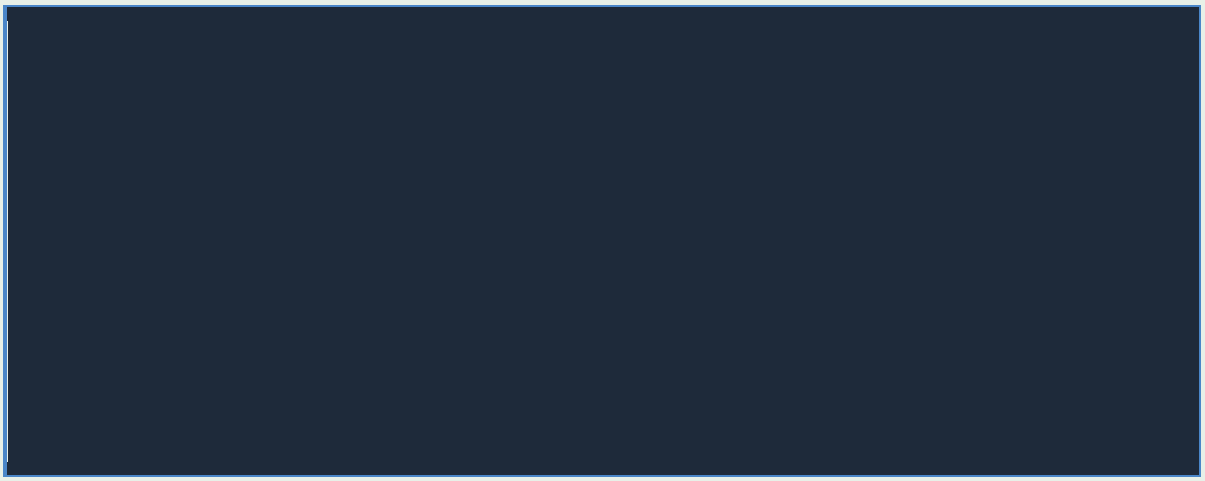
Write Python code using pandas to read a CSV file from disk and a Parquet file, then compare the row count and memory usage of each. Explain the output.

Answer:

Coding — Q3

Implement a timestamp-based CDC query in Python using a MySQL database. The script should fetch only rows updated since the last pipeline run, store the checkpoint, and print the results.

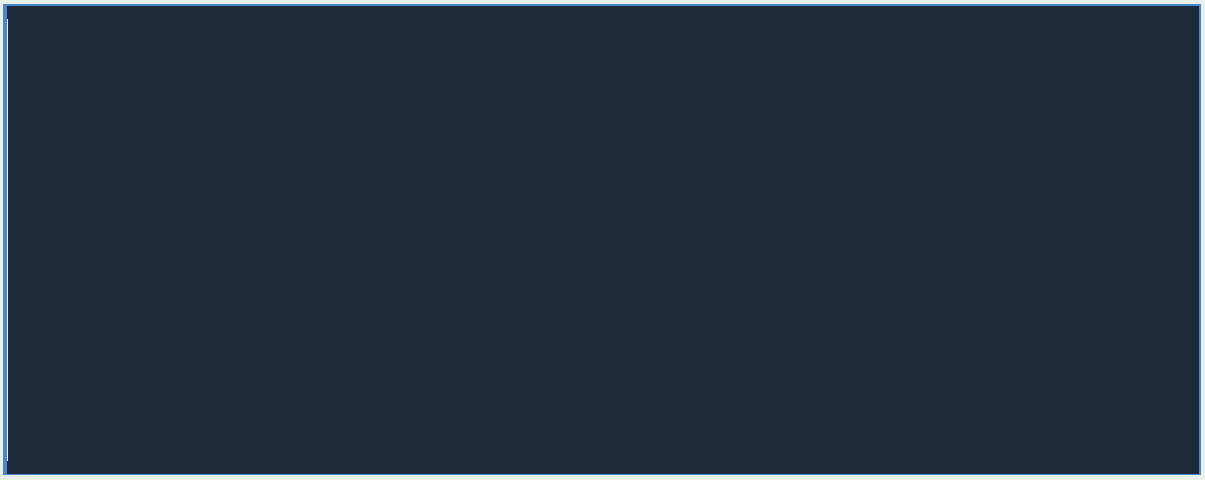
Answer:



Coding — Q4

Write a Python function to read multiple CSV files from a folder, convert each to Parquet format, and save them to an output directory. Include basic error handling.

Answer:



Section D — Database Scenario-Based Coding

Database — Q1

Scenario: You have a staging table staging.orders_cdc that contains CDC events (INSERT, UPDATE, DELETE) from your source system. Write a Snowflake SQL MERGE statement to apply these changes to your analytics.orders target table. Ensure that new records are inserted, existing records are updated, and deleted records are removed.

Answer:

Database — Q2

Scenario: Your data warehouse has a raw_orders table loaded from an ELT pipeline. Write a BigQuery SQL transformation (as a dbt-style model) to create a clean clean_orders table: standardise customer_name to uppercase, cast order_date to DATE, convert order_amount from USD to INR (rate: 83.0), and exclude orders with zero or negative amounts.

Answer:

Database — Q3

Scenario: An analyst notices the orders table in Redshift has duplicate rows for some order_ids after the nightly CDC load. Write a SQL query to (a) identify the duplicates, (b) count how many order_ids are affected, and (c) write a deduplication query that keeps only the most recently updated row per order_id.

Answer:

Unit 1 — Data Integration (ELT & ETL)

Interview-Base Unit Test— Warmup & Interviewer Levels

Level 1: Warmup Test

5 Multiple Choice Questions | Topic: Data Integration (ELT & ETL)

Q1. What does ELT stand for, and what is its key difference from ETL?

A) Extract, Load, Transform — transformation happens before loading
B) Extract, Load, Transform — transformation happens after loading inside the warehouse
C) Extract, Link, Transform — data is linked between sources first
D) Extract, Layer, Transfer — data is layered in the pipeline before transfer

✓ **Answer:** B) Extract, Load, Transform — transformation happens after loading inside the warehouse

Q2. Which of the following is the BEST reason to use batch processing over real-time streaming?

A) You need to detect credit card fraud the moment a transaction occurs
B) A GPS tracking app needs to update a rider's live location every 3 seconds
C) A payroll system processes employee salary calculations at the end of each month
D) A live sports scoreboard updates after every ball bowled

✓ **Answer:** C) A payroll system processes employee salary calculations at the end of each month

Q3. Which CDC method reads the database transaction log to capture all changes in real time?

A) Timestamp-Based CDC — queries rows where updated_at > last_run_time
B) Trigger-Based CDC — uses database triggers to write changes to an audit table
C) Log-Based CDC — reads the binlog (MySQL) or WAL (PostgreSQL) for all changes
D) Snapshot-Based CDC — takes a full table snapshot and compares differences

✓ **Answer:** C) Log-Based CDC — reads the binlog (MySQL) or WAL (PostgreSQL) for all changes

Q4. Which managed connector is open-source and best known for its flexibility with custom connectors?

A) Fivetran — commercial SaaS focused on enterprise reliability
B) Stitch — designed for simplicity and small teams
C) Airbyte — open-source with 350+ connectors and custom connector support
D) Debezium — a CDC tool that reads database transaction logs

✓ **Answer:** C) Airbyte — open-source with 350+ connectors and custom connector support

Q5. Which file format is BEST suited for large-scale analytics queries due to its columnar storage and excellent compression?

A) CSV — human-readable, universal support
B) Avro — row-based binary format ideal for streaming with Kafka
C) JSON — flexible schema, widely used in APIs
D) Parquet — columnar binary format with excellent compression (3–10x smaller)

✓ **Answer:** D) Parquet — columnar binary format with excellent compression (3–10x smaller)

Level 2: Interviewer Test

15 Intermediate–Advanced Questions | Descriptive · Scenario · Coding · Database

Section A — Descriptive Questions

Descriptive — Q1

Explain the difference between ETL and ELT. Why do modern cloud-based data stacks prefer ELT over traditional ETL?

Answer:

So the way I think about it — both ETL and ELT are about moving data from a source to a destination, but the key difference is *when* the transformation happens.

In ETL (Extract, Transform, Load), you pull the data, clean and reshape it on a separate transformation server, and only then load the polished data into the warehouse. The classic example would be a Talend job that runs overnight.

In ELT (Extract, Load, Transform), you first dump the raw data straight into the warehouse — say BigQuery or Snowflake — and *then* you run your SQL transformations right there inside the warehouse. Tools like dbt make this super clean.

The reason modern stacks prefer ELT is really about leverage. Cloud warehouses like BigQuery can spin up hundreds of parallel compute slots and process terabytes in minutes. So why maintain a separate transformation server when you can just use the warehouse's own engine? It's faster, more scalable, and frankly more transparent — the raw data is always preserved, so if a transformation is wrong, you just fix the SQL and rerun. No data is lost.

ETL still makes sense in regulated industries like banking, where you need to mask or tokenize sensitive data *before* it even enters the warehouse — compliance reasons, not performance ones.

Descriptive — Q2

Compare Batch Processing and Real-Time (Streaming) Processing. What factors should a data team consider when choosing between them?

Answer:

The simplest way I think about batch vs. real-time is: *how stale can your data afford to be?*

Batch processing collects data over a period — say hourly or daily — and processes it all at once. It's simpler to build, cheaper to run, and works perfectly for things like monthly payroll, end-of-day sales reports, or overnight inventory snapshots. Tools like Airflow + Spark are the go-to here.

Real-time (streaming) processes each event the moment it arrives — milliseconds of latency. It's essential for fraud detection, live GPS tracking, or ICU patient monitoring. The trade-off is complexity and cost — you're running Kafka, Flink, or Spark Structured Streaming 24/7.

The key factors I'd consider: (1) Latency tolerance — can the business wait hours, or does it need seconds? (2) Cost budget — streaming infrastructure is expensive. (3) Data volume — large historical batches are fine for batch; continuous small events need streaming. (4) Complexity tolerance — real-time systems are harder to debug and maintain.

There's also a middle ground called micro-batch — running jobs every 5 minutes using Spark Structured Streaming. It's a nice compromise when true real-time isn't needed but hourly batch is too slow.

Descriptive — Q3

What is Change Data Capture (CDC)? Describe the three main CDC methods and their trade-offs.

Answer:

CDC is basically the idea that instead of copying an entire database table every time your pipeline runs — which is incredibly expensive for large tables — you only capture the *rows that actually changed*: inserts, updates, and deletes.

There are three main approaches:

1. Timestamp-Based CDC: You query rows where updated_at > last_run_time. It's simple to implement with just SQL, but the big weakness is that it can't capture hard deletes — if a row is deleted from the source, there's no updated_at to filter on, so you miss it entirely.

2. Log-Based CDC: This reads the database's transaction log — the binlog in MySQL, or the WAL in PostgreSQL. It captures every single change, including deletes, in real time. Tools like Debezium use this method. It's the most reliable and complete, though it requires database-level access and a bit more infrastructure.

3. Trigger-Based CDC: Database triggers write every change to a separate audit table. It works without log access, but it adds overhead to every write operation on the source database — not ideal for high-traffic production systems.

In practice, log-based CDC is the gold standard, and tools like Fivetran and Debezium have made it much easier to set up without deep DBA expertise.

Descriptive — Q4

Explain what API Ingestion is. What are the key concepts an analyst must understand when pulling data from a SaaS tool's REST API?

Answer:

API ingestion is how you programmatically pull data from external SaaS tools — think Salesforce, Zendesk, HubSpot, or Google Ads — into your data warehouse. These tools expose REST APIs: essentially URLs you can hit with an HTTP request and get back structured JSON data.

As an analyst working with APIs, there are five key concepts I'd call out:

Authentication: APIs don't just hand out data to anyone. You'll typically authenticate using API Keys, OAuth 2.0 tokens, or Basic Auth credentials. Get this wrong and you get a 401 Unauthorized error.

Pagination: Large datasets are split across multiple pages. If you only fetch the first page, you're missing 90% of the data. You need to loop through all pages — checking for a next_page URL — until the response is empty.

Rate Limiting: APIs limit how many requests you can make per minute or hour. Exceed the limit and you get a 429 Too Many Requests error. Your pipeline needs to handle this gracefully — back off, wait, and retry.

Endpoints: Different URLs return different data. For example, /api/v2/tickets returns Zendesk tickets, while /api/v2/users returns user records. You need to know which endpoint has the data you need.

Webhooks: Instead of polling the API repeatedly, some tools let you register a URL and they push data to you the moment something changes. It's more efficient but requires you to host an endpoint that can receive events.

Section B — Scenario-Based Descriptive Questions

Scenario — Q1

Scenario: You are a data analyst at an e-commerce startup. Your company uses Shopify for orders, HubSpot for CRM, and a MySQL production database. Your manager asks you to build a single analytics view of customer orders. You have no backend engineering support. What approach would you recommend, and why?

Answer:

This is actually a really common situation for analysts at startups, and the good news is there's a clear answer: a managed connector like Fivetran or Airbyte.

Here's my thinking. Shopify and HubSpot both have pre-built connectors in Fivetran. With a few hours of configuration — no code at all — I can get both data sources syncing into BigQuery every hour. For the MySQL production database, Fivetran supports log-based CDC, so it will only capture what changed rather than doing a full dump every run.

Once all three sources are in BigQuery, I'd write a dbt model (ELT approach) to join them into a unified customer_orders view — matching HubSpot contacts to Shopify orders via email or customer ID, and enriching with any production MySQL data.

Why not build custom API extractors? Because without an engineering team, I'd spend weeks maintaining authentication, pagination, rate limit handling, and schema change bugs — time better spent on analysis. Fivetran handles all of that automatically.

The result: a clean, self-updating analytics table in BigQuery that my manager can query or connect to Looker for dashboards — with zero ongoing maintenance from my side on the ingestion layer.

Scenario — Q2

Scenario: A bank's nightly ETL job takes 4 hours to copy a 50-million-row customer accounts table from the core banking system into the data warehouse. The team is told to reduce this to under 10 minutes. What solution would you propose, and what are the risks?

Answer:

The root problem here is obvious — copying 50 million rows every night when only a fraction actually changed is massively inefficient. The solution is to implement CDC, specifically log-based CDC.

My proposal: deploy Debezium to read the core banking system's transaction log (WAL if it's PostgreSQL). Debezium will capture only the rows that were inserted, updated, or deleted since the last run — let's say 10,000 rows per day. That sync would take 2 minutes, not 4 hours.

In the warehouse, instead of overwriting the table, we'd use a MERGE (upsert) statement to apply the changes — updating matching rows and inserting new ones.

Now for the risks, because this is a bank and I want to be honest: Log-based CDC requires access to the database transaction log, which may need DBA sign-off and compliance review. There's also the risk of missing deletes if the CDC stream lags or goes down — we'd need monitoring and alerting on the CDC pipeline. Schema changes on the source table can break the CDC stream if not handled carefully.

Mitigation: Run the CDC pipeline with monitoring, maintain the full snapshot as a monthly fallback, and test the reconciliation between CDC-derived data and the source count before going live.

Scenario — Q3

Scenario: A food delivery company processes 1 million GPS location updates per minute from delivery riders. The analytics team wants to build a live rider tracking dashboard. A junior analyst suggests using a daily batch job. Do you agree? What would you recommend?

Answer:

I'd politely but firmly disagree with the daily batch suggestion here. The use case is live rider tracking — a customer is literally watching a dot move on a map. If the location is updated once a day, the product is broken. This is a textbook real-time streaming requirement.

What I'd recommend: The GPS events should be published to an Apache Kafka topic — Kafka is designed precisely for this kind of high-throughput, low-latency event stream (1 million events per minute is no problem). A stream processing layer — Kafka Streams or Apache Flink — would process each location update and write the latest position to a low-latency store (Redis or a real-time database like Firestore).

The dashboard would query this low-latency store, not the data warehouse, to get the current rider position.

Now — there's still a role for batch here. For historical analysis (average delivery time, heatmaps of popular pickup zones), we'd batch-load the Kafka events into BigQuery or Redshift at the end of the day. That's the lambda architecture: real-time layer for live tracking, batch layer for historical analytics.

The key insight I'd share with the junior analyst: real-time and batch aren't mutually exclusive. The question is always which layer serves which need — and live GPS tracking can never be served by batch.

Scenario — Q4

Scenario: Your company receives daily CSV sales files from 30 regional partners, uploaded to an S3 bucket by 8 AM. The pipeline loads them into Redshift for reporting. A

partner complains their data isn't showing up in dashboards. Walk through how you would debug this end-to-end.

Answer:

This is a debugging exercise, and I'd approach it layer by layer — starting from the source and moving toward the destination.

Step 1 — Did the file arrive? I'd check the S3 bucket for that partner's folder and confirm the file is present with today's date and a non-zero file size. If the file is missing, the problem is on the partner's side — I'd notify them.

Step 2 — Did the pipeline run? I'd check the Airflow (or whatever scheduler) logs for today's DAG run. Did it succeed, fail, or skip? If it failed, what was the error?

Step 3 — File format issue? CSV files from external partners are notoriously messy. I'd open the file and check: Is the delimiter correct? Is the encoding UTF-8 or something else (like Latin-1)? Are there extra header rows? Does the schema match what Redshift expects?

Step 4 — Load error in Redshift? I'd query the STL_LOAD_ERRORS system table in Redshift to see if specific rows were rejected during the COPY command — this often reveals type mismatches or null values in required columns.

Step 5 — Data made it in but dashboard filter issue? Sometimes the data loads correctly but the dashboard has a date filter that doesn't include today's data. I'd manually query the table with WHERE partner_id = X AND date = TODAY to confirm.

The fix depends on which step fails — but this systematic approach means I'm not guessing, I'm following the data trail from S3 to the dashboard.

Section C — Coding Questions

Coding — Q1

Write a Python script to call the Zendesk REST API, retrieve all open tickets with pagination, and store them in a list. Handle authentication with a Bearer token.

Answer:

The key things I'm keeping in mind here: auth header, handling the next_page loop for pagination, and collecting all tickets into a single list.

```
import requests
```

```
BASE_URL = "https://yourcompany.zendesk.com/api/v2"
API_TOKEN = "YOUR_ZENDESK_API_TOKEN"

headers = {
 "Authorization": f"Bearer {API_TOKEN}",
 "Content-Type": "application/json"
}

def fetch_all_open_tickets():
 tickets = []
 url = f"{BASE_URL}/tickets.json?status=open"

 while url:
 response = requests.get(url, headers=headers)

 if response.status_code == 429: # Rate limit hit
 print("Rate limited — waiting 60s")
 import time; time.sleep(60)
 continue

 response.raise_for_status() # Raise for 4xx/5xx errors
 data = response.json()
 tickets.extend(data.get("tickets", []))
 url = data.get("next_page") # None stops the loop

 return tickets

all_tickets = fetch_all_open_tickets()
print(f"Total tickets fetched: {len(all_tickets)}")
```

Why I structured it this way: The while url: loop keeps fetching as long as next_page is not None. I added a 429 handler because rate limiting is a real-world problem with Zendesk. Using raise_for_status() ensures we catch errors early rather than silently processing empty data.

Coding — Q2

Write Python code using pandas to read a CSV file from disk and a Parquet file, then compare the row count and memory usage of each. Explain the output.

Answer:

This is a great practical exercise to demonstrate why Parquet is preferred for large datasets.

```python
import pandas as pd

# Read CSV
df_csv = pd.read_csv("sales_data.csv", encoding="utf-8")

# Read Parquet
df_parquet = pd.read_parquet("sales_data.parquet")

# Row counts
print(f"CSV rows : {len(df_csv):,}")
print(f"Parquet rows: {len(df_parquet):,}")

# Memory usage in MB
csv_mem = df_csv.memory_usage(deep=True).sum() / 1024**2
parquet_mem = df_parquet.memory_usage(deep=True).sum() / 1024**2

print(f"CSV memory usage : {csv_mem:.2f} MB")
print(f"Parquet memory usage: {parquet_mem:.2f} MB")

# Schema comparison
print("\nCSV dtypes:")
print(df_csv.dtypes)

print("\nParquet dtypes:")
print(df_parquet.dtypes)
```

What I'd expect to see: Both have the same row count if the data is equivalent. Parquet often uses significantly less memory because it stores data with type-aware compression — integers as int32/int64, categories as dictionary-encoded. CSV loads everything as strings first, which inflates memory. Parquet also preserves schema (dtypes), so you won't see object dtype for numeric columns the way CSV sometimes does.

Coding — Q3

Implement a timestamp-based CDC query in Python using a MySQL database. The script should fetch only rows updated since the last pipeline run, store the checkpoint, and print the results.

Answer:

The trick here is persisting the last run timestamp so the next run picks up from where we left off — otherwise we'd always pull everything.

```
import mysql.connector
import json
from datetime import datetime

CHECKPOINT_FILE = "cdc_checkpoint.json"

def load_checkpoint():
 try:
 with open(CHECKPOINT_FILE) as f:
 return json.load(f)["last_run"]
 except FileNotFoundError:
 return "1970-01-01 00:00:00" # First run: pull all history

def save_checkpoint(ts):
 with open(CHECKPOINT_FILE, "w") as f:
 json.dump({"last_run": ts}, f)

def run_cdc():
 last_run = load_checkpoint()
 current_run = datetime.utcnow().strftime("%Y-%m-%d %H:%M:%S")

 conn = mysql.connector.connect(
 host="localhost", user="analyst",
 password="secret", database="source_db"
 )
 cursor = conn.cursor(dictionary=True)

 query = """
 SELECT order_id, customer_id, status, updated_at
 FROM orders
 WHERE updated_at > %s
 AND updated_at <= %s
 ORDER BY updated_at ASC
 """
 cursor.execute(query, (last_run, current_run))
 rows = cursor.fetchall()

 print(f"CDC pulled {len(rows)} changed rows since {last_run}")
 for row in rows:
 print(row)

 cursor.close()
```

```
 conn.close()

 save_checkpoint(current_run) # Advance checkpoint only on
success

run_cdc()
```

Key design decisions: I save the checkpoint only *after* a successful run. If the pipeline crashes mid-way, the checkpoint stays at the old value, so the next run will re-pull and nothing is missed. The AND updated_at <= current_run bound ensures we don't pull rows that get updated mid-query (avoiding a race condition).

Coding — Q4

Write a Python function to read multiple CSV files from a folder, convert each to Parquet format, and save them to an output directory. Include basic error handling.

Answer:

This pattern is really common in ELT pipelines — partners drop CSVs, we convert to Parquet for efficient warehouse loading.

```
import pandas as pd
import os
from pathlib import Path

def convert_csvs_to_parquet(input_dir: str, output_dir: str) ->
None:
 """
 Reads all CSV files from input_dir and converts
 each to Parquet format in output_dir.
 """
 Path(output_dir).mkdir(parents=True, exist_ok=True)
 csv_files = list(Path(input_dir).glob("*.csv"))

 if not csv_files:
 print(f"No CSV files found in {input_dir}")
 return

 success, failed = 0, []

 for csv_path in csv_files:
```

```
 try:
 df = pd.read_csv(csv_path, encoding="utf-8")

 # Build output path with .parquet extension
 out_path = Path(output_dir) /
csv_path.with_suffix(".parquet").name
 df.to_parquet(out_path, index=False, engine="pyarrow")

 print(f" Converted: {csv_path.name} → {out_path.name} "
 f"({len(df):,} rows)")
 success += 1

 except Exception as e:
 print(f" FAILED: {csv_path.name} — {e}")
 failed.append(csv_path.name)

 print(f"\nDone: {success} converted, {len(failed)} failed.")
 if failed:
 print("Failed files:", failed)

# Usage
convert_csvs_to_parquet(
 input_dir="data/raw_csv",
 output_dir="data/parquet"
)
```

The try/except per file is important — if one partner's CSV is malformed, we don't want it to kill the entire batch. We log the failure and continue. Using pyarrow as the Parquet engine gives better type inference and performance than fastparquet for most use cases.

Section D — Database Scenario-Based Coding

Database — Q1

Scenario: You have a staging table staging.orders_cdc that contains CDC events (INSERT, UPDATE, DELETE) from your source system. Write a Snowflake SQL MERGE statement to apply these changes to your analytics.orders target table. Ensure that new records are inserted, existing records are updated, and deleted records are removed.

Answer:

This is the classic CDC merge pattern — the foundation of any incremental pipeline. Here's how I'd approach it in Snowflake:

```
-- ─────────────────────────────────────────────────────────────
-- CDC MERGE: Apply INSERT / UPDATE / DELETE from staging
-- ─────────────────────────────────────────────────────────────

MERGE INTO analytics.orders AS target
USING (
 -- De-duplicate: keep only the latest CDC event per order_id
 SELECT *
 FROM (
 SELECT *,
 ROW_NUMBER() OVER (
 PARTITION BY order_id
 ORDER BY event_timestamp DESC
 ) AS rn
 FROM staging.orders_cdc
 WHERE DATE(event_timestamp) = CURRENT_DATE
 )
 WHERE rn = 1
) AS source
ON target.order_id = source.order_id

-- Update existing records (INSERT or UPDATE events)
WHEN MATCHED AND source.operation IN ('INSERT', 'UPDATE') THEN
 UPDATE SET
 target.customer_id = source.customer_id,
 target.order_status = source.order_status,
 target.order_amount = source.order_amount,
 target.updated_at = source.event_timestamp

-- Remove deleted records
WHEN MATCHED AND source.operation = 'DELETE' THEN
 DELETE

-- Insert brand-new records
WHEN NOT MATCHED AND source.operation != 'DELETE' THEN
 INSERT (order_id, customer_id, order_status, order_amount,
updated_at)
 VALUES (source.order_id, source.customer_id,
 source.order_status, source.order_amount,
 source.event_timestamp);
```

Why the de-duplication subquery? If the staging table gets multiple CDC events for the same order_id (e.g., an UPDATE followed immediately by another UPDATE), we only want to apply the *latest* one. The ROW_NUMBER() window function handles this cleanly. Without it, the MERGE might fail on duplicate source rows.

Database — Q2

Scenario: Your data warehouse has a raw_orders table loaded from an ELT pipeline. Write a BigQuery SQL transformation (as a dbt-style model) to create a clean clean_orders table: standardise customer_name to uppercase, cast order_date to DATE, convert order_amount from USD to INR (rate: 83.0), and exclude orders with zero or negative amounts.

Answer:

This is exactly the ELT pattern — raw data is already in the warehouse, now I'm transforming it with SQL. In dbt, this would live in a models/ file.

```
-- ─────────────────────────────────────────────────────────
-- dbt model: models/clean/clean_orders.sql
-- ─────────────────────────────────────────────────────────

{{
 config(
 materialized = 'table',
 schema = 'analytics'
 )
}}

SELECT
 order_id,
 customer_id,

 -- Standardise: trim whitespace + convert to uppercase
 UPPER(TRIM(customer_name)) AS customer_name,

 -- Type cast: ensure order_date is a proper DATE
 CAST(order_date AS DATE) AS order_date,

 -- Currency conversion: USD → INR at fixed rate 83.0
 ROUND(order_amount * 83.0, 2) AS order_amount_inr,
```

```
 -- Preserve original USD amount for auditability
 order_amount AS order_amount_usd,

 -- Categorise order size for downstream reporting
 CASE
 WHEN order_amount >= 1000 THEN 'High Value'
 WHEN order_amount >= 100 THEN 'Mid Value'
 ELSE 'Low Value'
 END AS order_category,

 CURRENT_TIMESTAMP() AS dbt_loaded_at

FROM {{ source('raw', 'raw_orders') }}

-- Exclude invalid / zero amount orders
WHERE order_amount > 0
 AND customer_id IS NOT NULL
 AND order_date IS NOT NULL;
```

A few things worth noting: I kept order_amount_usd alongside the converted INR amount — preserving the original is a best practice for auditability. The order_category CASE column adds business context for dashboards without any extra joins. In a real dbt project, the {{ source() }} macro would reference the schema and table registered in sources.yml.

Database — Q3

Scenario: An analyst notices the orders table in Redshift has duplicate rows for some order_ids after the nightly CDC load. Write a SQL query to (a) identify the duplicates, (b) count how many order_ids are affected, and (c) write a deduplication query that keeps only the most recently updated row per order_id.

Answer:

Duplicates in a CDC pipeline are actually a really common issue — they usually happen when the pipeline retries and loads the same batch twice. Here's my three-part approach:

```
-- ____________________________________________________
-- PART A: Identify duplicate order_ids
-- ____________________________________________________
SELECT
 order_id,
```

```
 COUNT(*) AS row_count
FROM analytics.orders
GROUP BY order_id
HAVING COUNT(*) > 1
ORDER BY row_count DESC;



-- ─────────────────────────────────────────────────────────────
-- PART B: How many order_ids are affected?
-- ─────────────────────────────────────────────────────────────
SELECT COUNT(*) AS duplicate_order_ids
FROM (
 SELECT order_id
 FROM analytics.orders
 GROUP BY order_id
 HAVING COUNT(*) > 1
);



-- ─────────────────────────────────────────────────────────────
-- PART C: Deduplicate — keep most recently updated row
-- (Redshift approach using CREATE TABLE AS)
-- ─────────────────────────────────────────────────────────────

-- Step 1: Create deduplicated copy
CREATE TABLE analytics.orders_deduped AS
SELECT *
FROM (
 SELECT *,
 ROW_NUMBER() OVER (
 PARTITION BY order_id
 ORDER BY updated_at DESC -- Keep latest row
 ) AS rn
 FROM analytics.orders
)
WHERE rn = 1;

-- Step 2: Swap the tables (in a transaction for safety)
BEGIN;
 DROP TABLE analytics.orders;
 ALTER TABLE analytics.orders_deduped RENAME TO orders;
COMMIT;
```

Why ROW_NUMBER() over DISTINCT? Because two duplicate rows can have the same order_id but different updated_at timestamps (if one is a retry from a later batch). We want the *latest* one, not

just any one. Running this in a transaction means if the rename fails, we still have the original table intact — critical for a production system.